

Bài 4:

Đa hình - Polymorphism

Đặt vấn đề

- Đại lý xe ô tô cần quản lý nhiều loại xe khác nhau có nhiều đặc tính khác nhau nhưng mỗi loại đều có 1 giá bán và số lượng tồn kho.
- Một nhà sinh vật học cần quản lý danh sách các loài động vật khác nhau với những đặc tính khác nhau nhưng tất cả chúng là động vật.

Mục tiêu

- Làm thế nào tổ chức một hệ thống thông tin phù hợp cho tất cả các loại?
- Vì các lớp trong hệ thống phân cấp (kế thừa) thường chia sẻ các phương thức và thuộc tính chung và chúng ta muốn tận dụng để làm cho các thuật toán đơn giản hơn.

Polymorphism

- **Tính đa hình (Polymorphism):** là khả năng một đối tượng có thể thực hiện một phương thức theo nhiều cách khác nhau.

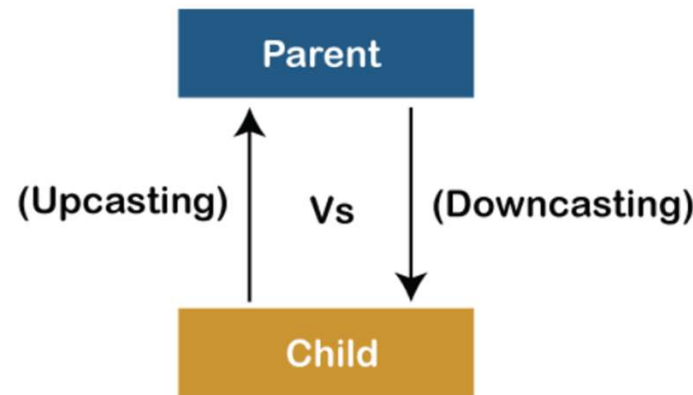


Polymorphism

- Trong Java, chúng ta sử dụng nạp chồng phương thức (method overloading) và ghi đè phương thức (method overriding) để có tính đa hình.
- Việc thực hiện phương thức nào sẽ được thực hiện lúc chạy chương trình (runtime) không phải lúc biên dịch (compile time)

Polymorphism

Upcasting và Downcasting: Là quá trình ép kiểu (casting) qua lại giữa 2 đối tượng thuộc lớp cha và lớp con.



Polymorphism

Upcasting: khi biến tham chiếu của lớp cha tham chiếu tới đối tượng của lớp con, thì đó là Upcasting

VD:

Ví dụ: upcasting

```
4 class Parent {  
5     String name;  
6  
7     void showMessage()  
8     {  
9         System.out.println("Parent method is called");  
10    }  
11 }
```

```
4 class Child extends Parent {  
5  
6     int age;  
7     void showMessage() {  
8  
9     }  
10 }
```

```
5     public static void main(String args[]) {  
6  
7         Parent p = new Child(); //Upcasting  
8     }
```


Ví dụ: downcasting

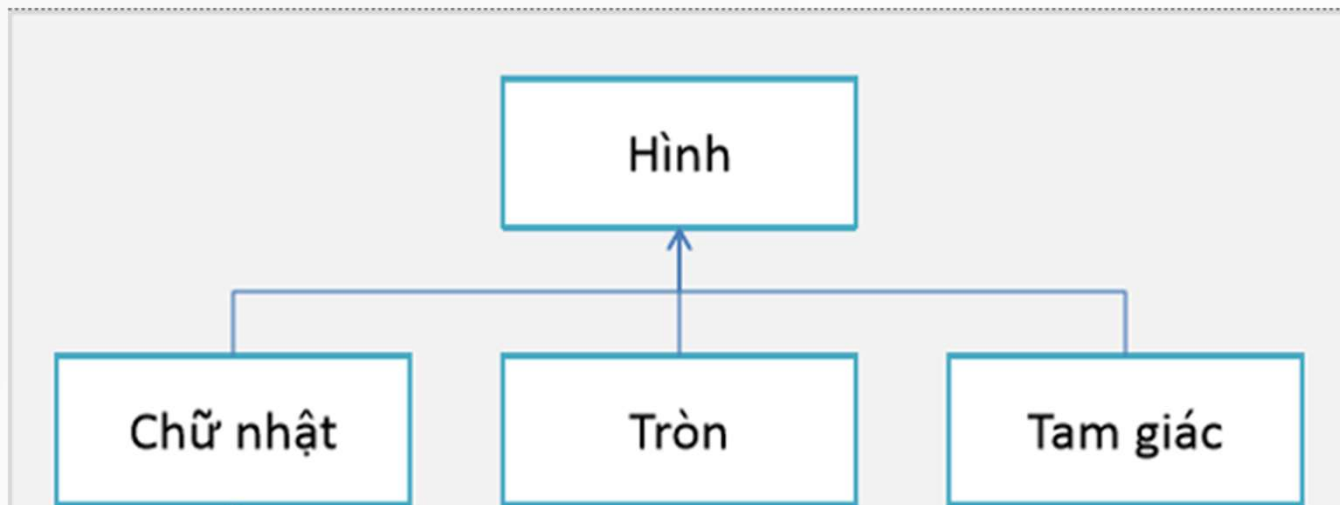
```
5 public static void main(String args[]) {  
6  
7     Parent p = new Child();  
8  
9     p.name = "Peter";  
10  
11     //Child c = new Parent(); // compile-time error  
12  
13     Child c = (Child)p;    // Downcasting: parent -> child  
14  
15     c.showMessage();  
16  
17 }
```

Lớp trừu tượng và phương thức trừu tượng

➤ Lớp trừu tượng (**Abstract class**): là lớp có phương thức chưa được xác định.

Ví dụ 1 : Đã là hình (hình học) thì chắc chắn là có chu vi và diện tích nhưng chưa xác định được cách tính mà phải là một hình cụ thể như hình tròn, vuông, ... mới có thể xác định cách tính

Lớp trừu tượng và phương thức trừu tượng



Lớp trừu tượng và phương thức trừu tượng

- Từ khóa **abstract** được dùng để định nghĩa lớp và phương thức trừu tượng
- Phương thức trừu tượng không có phần thân hàm
- Lớp chứa phương thức trừu tượng là lớp trừu tượng.
- Lớp trừu tượng có thể định nghĩa các phương thức cụ thể và các thuộc tính
- Không thể sử dụng **new** để tạo đối tượng cho lớp trừu tượng

Lớp trừu tượng và phương thức trừu tượng

```
abstract public class MyClass{  
    abstract public type MyMethod();  
}
```

Sử dụng từ khóa
abstract để định
nghĩa lớp và
phương thức trừu
tượng

```
abstract public class SinhVien{  
    abstract public double getDiemTB();  
}
```

```
abstract public class Hinh{  
    abstract public double getChuVi();  
    abstract public double getDienTich();  
}
```

Lớp trừu tượng và phương thức trừu tượng

```
abstract public class SinhVien{  
    public String hoTen;  
    abstract public double getDiemTB();  
}
```

```
public class SinhVienIT extends SinhVien{  
    public double diemJava;  
    public double diemCss;  
    @Override  
    public double getDiemTB() {  
        return (2 * diemJava + diemCss)/3;  
    }  
}
```

```
public class SinhVienBiz extends SinhVien {  
    public double keToan;  
    public double marketing;  
    public double banHang;  
    @Override  
    public double getDiemTB() {  
        return  
            (keToan + marketing + banHang)/3;  
    }  
}
```

Interface

- Interface là cơ chế hỗ trợ việc trừu tượng hoá trong java và thực hiện đa kế thừa.
- Một **Interface trong Java** là một bản thiết kế của một lớp.
- Interface là một tập các phương thức mà một class “*buộc phải định nghĩa*” khi triển khai.

Interface

- Interfaces làm nhiệm vụ khai báo các phương thức mà class cần phải có, nhưng không có phần “thân hàm” – (function body)
- Khi một class implements từ interface nào đó thì bắt buộc phải override tất cả các phương thức đã được khai báo trong interface tương ứng

Interface

- Có thể Interfaces giống như abstract classes, nhưng chúng khác nhau ở chỗ:
 - Một lớp con – subclass chỉ có thể thừa kế bởi **duy nhất** một lớp cha – superclass
 - Tuy nhiên, **có thể sử dụng nhiều** interfaces để làm tiêu chuẩn cho việc định nghĩa một class
 - Một interface không thể có thể hiện cụ thể, tức là **không thể tạo ra đối tượng của interface thông qua quản tử new**
Comparable list= **new** Comparable(); //-- *Báo lỗi, vì Comparable là một interface*

Interface

- Nhưng lại có thể khai báo 1 đối tượng là interface

Comparable list; //-- OK

- Cũng có thể tham chiếu đến đối tượng của class đã triển khai từ interface tương ứng:

Comparable list= **new** MySortClass();

//-- OK, trong trường hợp MySortClass implements từ Comparable

- Interfaces có thể chứa các khai báo ở dạng phương thức – method hoặc các hằng

```
public interface Rotatable {  
    void rotate(double theta);  //--- List reqd methods  
    double MAX_ROTATE= 360;  
} //--- Implicitly final
```

Interface

- Interface cho phép thừa kế, tức là có thể tạo ra 1 interface mới, thừa kế từ 1 interface đã có

```
3 public interface InterfaceA {  
4  
5     public void methodA();  
6 }
```

```
3 public interface InterfaceB extends InterfaceA{  
4  
5     public void methodB();  
6 }
```

Implement 1 class từ interface

```
3 public interface InterfaceA {  
4  
5     public void methodA();  
6 }
```

```
3 public interface InterfaceB extends InterfaceA {  
4  
5     public void methodB();  
6 }
```

```
3 public class ClassAB implements InterfaceB {  
4  
5     @Override  
6     public void methodA() {  
7  
8     }  
9     @Override  
10    public void methodB() {  
11  
12    }  
13 }
```

Đa kế thừa trong java

```
3 public interface InterfaceA {  
4  
5     public void methodA();  
6 }
```

```
3 public interface InterfaceB{  
4  
5     public void methodB();  
6 }
```

```
3 public class ClassAB implements InterfaceA, InterfaceB{  
4  
5     @Override  
6     public void methodA() {  
7  
8     }  
9     @Override  
10    public void methodB() {  
11  
12    }  
13 }
```

Đa kế thừa trong java

```
3 public class ClassAB extends ExampleClass implements InterfaceA, InterfaceB {  
4  
5     @Override  
6     public void methodA() {  
7  
8     }  
9     @Override  
10    public void methodB() {  
11  
12    }  
13 }
```